Implementation and Analysis of a Cryptographic Protocol on an Elliptic Curve

Chabane Theo - Mathieu Jay - Alexy Carrere

27/03/2025

1 Introduction

ECC (Elliptic Curve Cryptography) is a public-key cryptography method based on elliptic curves over finite fields. .

Public-key cryptography (or asymmetric cryptography) is a cryptographic system using a pair of keys: a **public key**, shared with everyone, for encrypting or verifying signatures, and a **private key**, kept secret, for decrypting or signing. It is widely used in the real world and in many fields: Computer security, Finance, Cryptocurrencies, Digital identity, Telecommunications...

ECC has become popular due to its security and efficiency. Before its rise, most public-key algorithms were based on **RSA** and **Diffie-Hellman**, which are alternative cryptographic systems relying on modular arithmetic. However, **ECC** achieves a level of security equivalent to that of other public-key cryptography methods, but with shorter keys. This characteristic results in faster encryption and decryption operations that require fewer resources.

It is important to note that current cryptosystems used in widely deployed protocols such as **TLS**, **PGP**, and **SSH** partly rely on elliptic curves, not to mention their essential role in **Bitcoin** and other cryptocurrencies. Thus, **ECC** is a fundamental component of the security of modern communications and digital transactions.

The objective of this applied research project is to deepen the understanding of the mathematical concepts underlying elliptic curve cryptography (**ECC**) and to implement them in a cryptographic protocol using Python. This includes the implementation of fundamental principles, performance analysis of the developed code, and comparison of **ECC** with other encryption methods.

Once the concepts around **ECC** and its cryptography have been explained, we will apply the previous concepts to a real-world case: **Bitcoin**. We will explain the role of **ECC** in **Bitcoin**, the particularities of its elliptic curve, as well as simulate the creation of a **Bitcoin** address, the signing, and the verification of a fictitious transaction.

Contents

1 Introduction **1**

2 Implementation **3**

3 Definition of an Elliptic Curve over $\mathbb{R}$ **3**
3.1 Group Theory Basics . . . 3
3.2 Addition over $\mathbb{R}$ . . . 4
3.2.1 Addition of two points over $\mathbb{R}$ from a geometric perspective . . . 4
3.2.2 Addition of two points over $\mathbb{R}$ from an arithmetic perspective . . . 4
3.3 Scalar Multiplication over $\mathbb{R}$ . . . 5

4 Elliptic Curve over Finite Fields **5**
4.1 Addition over $\mathbb{F}_p$ . . . 6
4.1.1 Addition of two points over $\mathbb{F}_p$ from a geometric perspective . . . 6
4.1.2 Addition of two points over $\mathbb{F}_p$ from an arithmetic perspective . . . 6
4.2 Multiplication over $\mathbb{F}_p$ . . . 7

5 Cryptographic Protocol **7**
5.1 **ECDH** and **AES** . . . 7
5.1.1 **Elliptic Curve Diffie-Hellman** (**ECDH**) . . . 7
5.1.2 AES Encryption in GCM Mode . . . 8

6 Critical Analysis **8**
6.1 Results and Comparison with RSA . . . 8
6.2 Limitations of Our Code . . . 9
6.3 The Discrete Logarithm Problem on an Elliptic Curve . . . 9
6.4 Potential Vulnerabilities of ECC . . . 10

7 (Bonus) The Role of ECC in Bitcoin **11**
7.1 Cryptographic Use of Bitcoin . . . 11
7.1.1 Signature . . . 11
7.1.2 Address Generation . . . 11
7.2 Choice of the **secp256k1** Curve . . . 11
7.3 Implementation . . . 12
7.4 Security of Software Wallets . . . 13

8 Conclusion **13**

2 Implementation

For the implementation we will use Python along with the following libraries:

- **Random**: To generate random points
- **Numpy**: To generate and manipulate numerical arrays
- **Matplotlib**: To plot curves
- **OS/Sys**: To facilitate file navigation
- **Hashlib/base58**: Hashing and encoding of **Bitcoin** addresses
- **Cryptography**: For the benchmark between **RSA** and **ECDH**
- **Pytest/Unittest**: To test our functions (addition, scalar multiplication, etc.)

In our code we created an abstract class **Curve** that contains the functions and attributes common to both types of curves; the classes corresponding to the three types of curves inherit from it. A **Point** class to represent points on the curve and a **Person** class to simulate people during key and message exchange with **ECDH**.

3 Definition of an Elliptic Curve over $\mathbb{R}$

An elliptic curve over $\mathbb{R}$ is defined as follows:

$$y^2 = x^3 + ax + b \tag{1}$$

An elliptic curve must not be singular to ensure that the point addition operation works correctly and that the curve has an abelian group structure. To verify that the curve is not singular we use:

$$4a^2 + 27b^2 \neq 0 \tag{2}$$

3.1 Group Theory Basics

The point at infinity on an elliptic curve, denoted ω, is an important concept in algebraic geometry and cryptography. It serves as the neutral element for group operations on the points of the elliptic curve.

Elliptic curves have the particularity of always representing a so-called **abelian** group. This notion allows testing the generation of the curve to ensure that it is indeed elliptic.

A group is a set denoted G, represented here as the set of points on the elliptic curve, for which an operation denoted $+$ has been defined and which satisfies 4 rules:

- **Closure**: Any operation on two elements, represented here by two points, has its result in G:

$$\forall (a, b) \in G^2, \quad a + b \in G \tag{3}$$

- **Associativity**: Regardless of the order in which the elements are computed, the result is not affected:

$$\forall (a, b, c) \in G^3, \quad a + (b + c) = (a + b) + c \tag{4}$$

- **Neutral element**: The set G must have a so-called "neutral" element, here represented by the point at infinity ω :

$$\forall (a) \in G, \quad a + \omega = a \tag{5}$$

- **Inverse**: Every element in the set G must also have its inverse in said set:

$$\forall (a) \in G, \exists b \in G| \quad a + b = \omega \tag{6}$$

For a group to be considered abelian there is a 5th rule to satisfy:

- **Commutativity**: The operation on elements must be commutative. That is, one can swap the positions of two factors without affecting the result:

$$\forall (a, b) \in G, \quad a + b = b + a \tag{7}$$

To be able to use cryptographic algorithms on elliptic curves we will need to implement two mathematical operations on them: addition and multiplication.

3.2 Addition over $\mathbb{R}$

3.2.1 Addition of two points over $\mathbb{R}$ from a geometric perspective

The addition or sum of points on an elliptic curve is done by taking the line passing through two points P and Q; this line will intersect a third point R of the curve. We take the point -R which is the opposite of point R with respect to the x-axis; the point -R is the sum of points P and Q.

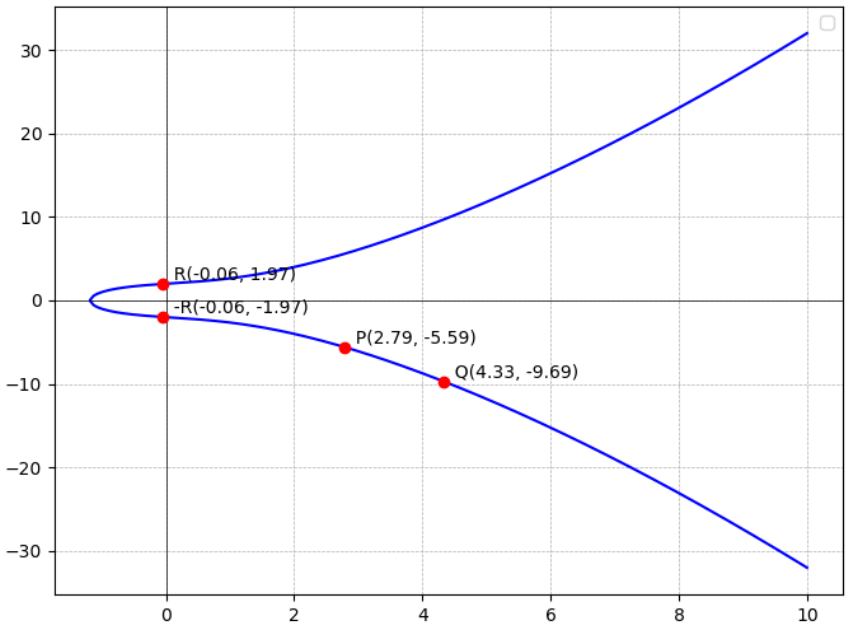

Figure 1: $y^2 = x^3 + 2x + 4$

3.2.2 Addition of two points over $\mathbb{R}$ from an arithmetic perspective

To compute the sum of points P and Q there are two formulas depending on whether the points are equal or not; in both cases we compute the slope before computing point R.

If $P \neq Q$, then

$$m = \frac{y_P - y_Q}{x_P - x_Q} \tag{8}$$

$$x_R = m^2 - x_P - x_Q \tag{9}$$

$$y_R = m(x_P - x_R) - y_P \tag{10}$$

If $P = Q$, then

$$m = \frac{3x_P^2 + a}{2y_P} \tag{11}$$

$$x_R = m^2 - 2x_P \tag{12}$$

$$y_R = m(x_P - x_R) - y_P \tag{13}$$

In addition to these algorithms we also had to implement the handling of certain special cases.

- **P** $= \omega$ **or Q** $= \omega$: We cannot draw any curve since ω is not representable. The addition simply returns the point that is not ω.
- **P = -Q**: The line is vertical and does not intersect any third point; there is no point R, so we return the point at infinity.

3.3 Scalar Multiplication over $\mathbb{R}$

Scalar multiplication plays a crucial role in elliptic curve cryptography since it is what allows generating public keys from private keys.
It consists of repeating additions of point P n times: first we add point P to itself, then we add it to the point given by the first addition, etc.

However, to make this operation faster, we use the *Double and Add* algorithm, which optimizes this scalar multiplication by using the decomposition into powers of two of n. Rather than performing $n - 1$ successive additions, the algorithm doubles the points at each step and adds P only when the corresponding bit of n is 1.
If n equals 1 we simply return point P.

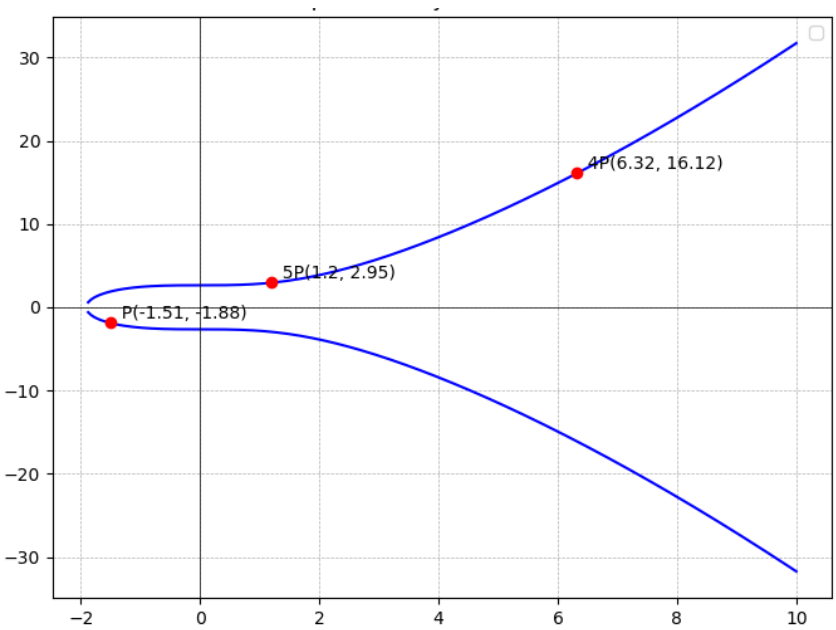

Figure 2: Computing point 5P from 4P

4 Elliptic Curve over Finite Fields

Elliptic curves over finite fields or $\mathbb{F}_p$ share common points with curves over $\mathbb{R}$ but they have several major differences. They are defined by

$$y^2 \equiv x^3 + ax + b \pmod{p} \tag{14}$$

Curves over $\mathbb{F}_p$ follow the same group principles as those over $\mathbb{R}$

The modulo p must be applied to all operations performed with them. Unlike curves over $\mathbb{R}$ curves over a finite field are not represented as a continuous curve but as a discrete cloud of unconnected points. This difference is due to the nature of the members on which we work.

The number of points on the graphical representation is called the order of the curve; it depends on a, b, and p. The order of an elliptic curve over $\mathbb{F}_p$ is given by *Hasse's theorem*, which establishes the following bound:

$$|\#E(\mathbb{F}_p) - (p+1)| \leq 2\sqrt{p}$$

This means that the number of points N satisfies:

$$p + 1 - 2\sqrt{p} \leq N \leq p + 1 + 2\sqrt{p}$$

Hasse's theorem implemented in Python gives us the minimum and maximum bounds of the curve's order. We then iterate through all values between these bounds, stopping at the first one that equals the number of points on the curve, providing a double verification (if the number is not within Hasse's bounds there is an error).

When computing the value of y we cannot simply take the square root of y^2 given by the curve's definition because, unlike real numbers, the square root of a number modulo p is not always defined. We therefore use the **Tonelli-Shanks** algorithm which efficiently solves quadratic equations modulo a prime number p and thus finds y.

4.1 Addition over $\mathbb{F}_p$

4.1.1 Addition of two points over $\mathbb{F}_p$ from a geometric perspective

In $\mathbb{F}_p$ there is no continuous curve nor visible lines. We can only see isolated points and addition computations are done purely through modular arithmetic.

4.1.2 Addition of two points over $\mathbb{F}_p$ from an arithmetic perspective

Addition follows the same logic as over $\mathbb{R}$ except that all computations are modular; for modular multiplication and division Python's *pow()* function suffices.

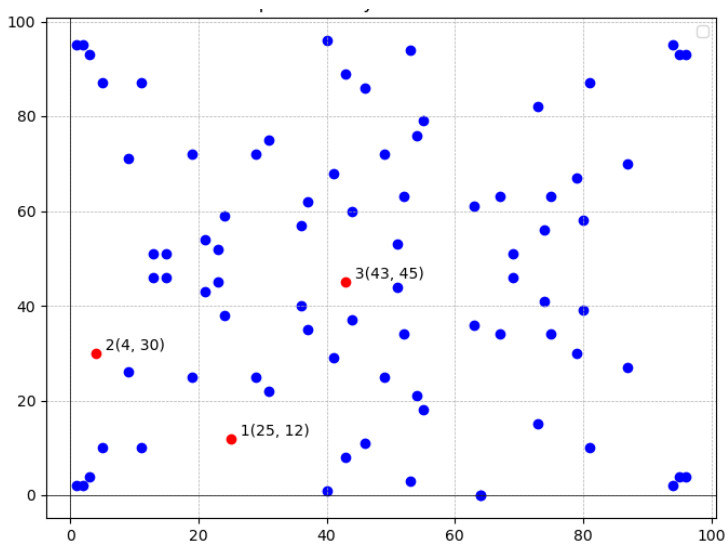

Figure 3: Addition on a $\mathbb{F}_p$ curve

4.2 Multiplication over $\mathbb{F}_p$

Multiplication follows the same arithmetic logic as multiplication over $\mathbb{R}$ but using the addition defined over $\mathbb{F}_p$.

5 Cryptographic Protocol

The main objective of **ECDH** (**Elliptic Curve Diffie-Hellman**) is to securely exchange a secret key between two parties (Alice and Bob) over an insecure channel. This shared key then allows encrypting communications with a symmetric algorithm, which is faster for large amounts of data. By relying on the difficulty of the **Diffie-Hellman** problem on elliptic curves, **ECDH** offers a level of security equivalent to other methods like **RSA**, but with shorter keys and more efficient operations. This protocol is therefore particularly suited for establishing a shared secret securely.

To encrypt a message on an elliptic curve we must generate a cyclic subgroup. This is a group of points that follow each other and contains ω (we add each point from the previous one until we reach ω). Its order is necessarily a divisor of the elliptic curve's order.

To generate this subgroup we must first find a suitable initial point; for this we first generate the desired subgroup order. The subgroup order n is the largest prime factor of the curve's order.

To find the starting point of the cyclic group we first compute the cofactor h from the curve's order N and the found subgroup order n.

$$h = N \div n \tag{15}$$

To find the starting point, we multiply a random point on the curve by the cofactor h. If the resulting point is the point at infinity, we choose a new random point.

We then start from this generator point, called G, to generate a cyclic subgroup by multiplying it with itself until we reach the point at infinity.

5.1 ECDH and AES

5.1.1 Elliptic Curve Diffie-Hellman (ECDH)

The **ECDH** protocol relies on elliptic curve arithmetic. Each user generates a private key d chosen randomly in the interval $[1, n-1]$, where n is **the order of the cyclic subgroup** found previously. The **public key** is obtained by **scalar multiplication**:

$$P = d \times G \tag{16}$$

where G is the **generator point of the curve** found previously.
The key exchange works as follows:

- Alice generates her **private key** d_A and her **public key** $P_A = d_A \times G$.
- Bob generates his **private key** d_B and his **public key** $P_B = d_B \times G$.
- After exchanging public keys, each can compute the **shared secret key**:

$$S = d_A \times P_B = d_B \times P_A \tag{17}$$

The equality comes from the fact that scalar multiplication is associative.

5.1.2 AES Encryption in GCM Mode

AES is a symmetric encryption algorithm, meaning it uses a single key to encrypt and decrypt the message. Here, we use the secret key obtained after the key exchange between Alice and Bob, then transform it into a 256-bit **AES** key using **HKDF** (*HMAC-based Key Derivation Function*).
HKDF produces a pseudo-random key from the **secret key**, which strengthens its security.

Once this derived key is obtained, **AES** can then encrypt the message. We use here the **GCM** mode of **AES**, which not only encrypts the data but also generates an **authentication tag**. This tag allows the recipient to verify that the message **has not been modified** in transit.

The result of the encryption is therefore:

- An **encrypted text**.

- An **authentication tag**.

To decrypt the message, Bob uses **the same secret key** to convert the encrypted text back into a readable message. He also verifies that the authentication tag is valid before accepting the message.

6 Critical Analysis

6.1 Results and Comparison with RSA

To demonstrate the efficiency of the **ECDH** algorithm, we compared it to **RSA** at an **equivalent security level**. In this case, a **128-bit security** level corresponds to a **3072-bit** key for **RSA** versus **256 bits** for **ECDH**.

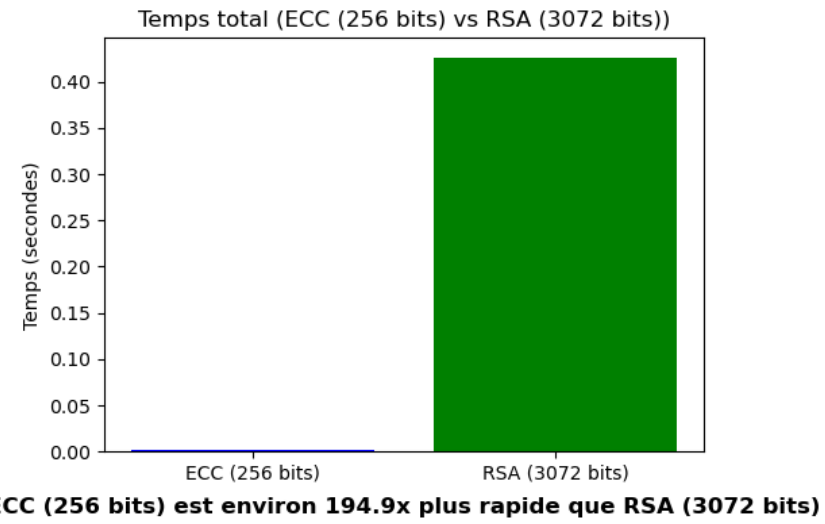

Figure 4: Comparative evaluation of total processing time between RSA and ECDH (key generation, encryption, and decryption)

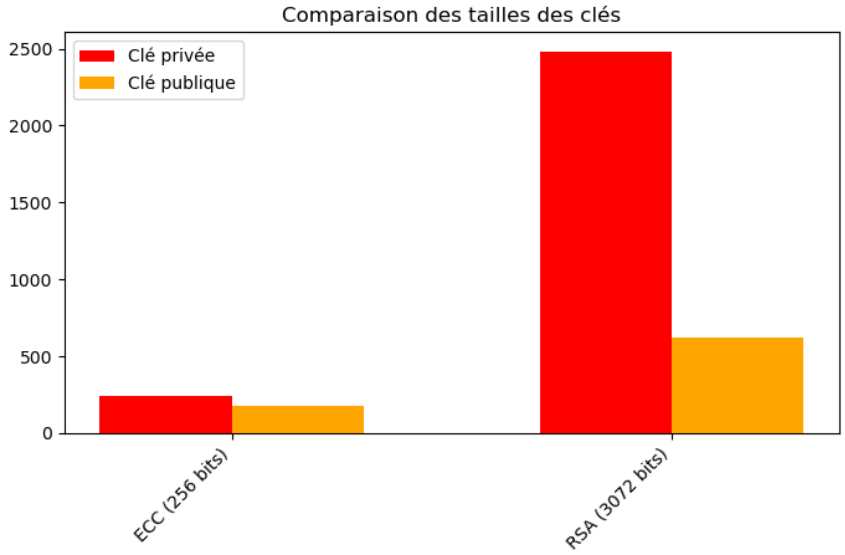

Figure 5: Comparison of public and private key sizes between RSA 3072 bits and ECDH 256 bits

We observe that at an equivalent security level, **ECDH** is significantly faster than **RSA**, while using keys of much smaller size. This result confirms what was presented in the introduction: **ECDH** achieves security comparable to that of classical algorithms like **RSA**, while offering superior cryptographic performance thanks to shorter keys and lighter operations.

6.2 Limitations of Our Code

Despite a correct implementation of the **ECDH** protocol, our code has certain limitations, mainly related to the size of the modulus p. Indeed, the larger p is, the more costly the computation of the elliptic curve's order becomes in terms of time.

This complexity is mainly due to the absence of an implementation of **Schoof's** algorithm. Consequently, we merely use **Hasse's** theorem to bound the curve's order, then explicitly count all points belonging to the curve within this interval, which is particularly inefficient when p becomes large.

Here is a concrete example obtained with a curve defined over $\mathbb{F}_{5791}$:

```
Order computation time:  11.536738157272339
Total time:  11.536853551864624
```

We observe that this execution time is almost entirely due to the order computation. This creates a real limitation to our implementation.

6.3 The Discrete Logarithm Problem on an Elliptic Curve

Elliptic curve cryptography relies on the difficulty of the **discrete logarithm problem** (**DLP**) in the group of points of an elliptic curve defined over a finite field. Rather than solving a classical exponential equation of the form

$$g^x \equiv h \pmod{p},$$

the problem on an elliptic curve consists of finding the secret integer k in the equation

$$Q = kG,$$

where:

- G is a public base point on the curve,
- k is a secret integer,

- Q is the result of the scalar multiplication of G by k.

Computing $Q = kG$ using the double-and-add algorithm is efficient, even for large values of k.

So why is finding k difficult?

1. **Complex inversion:** Recovering k from Q and G (i.e., solving the equation $Q = kG$ for k) is a problem for which no efficient polynomial-time algorithm is known.

2. **Impact of group size:** Security relies on the fact that, for a sufficiently large group of points (defined by Hasse's theorem), inverting the scalar multiplication computation becomes practically impossible in a reasonable time.

Conclusion: The difficulty of recovering the secret integer k from $Q = kG$ is what guarantees the security of cryptographic protocols based on elliptic curves, such as the **Diffie-Hellman** key exchange on elliptic curves (**ECDH**) and the **ECDSA** digital signature.

6.4 Potential Vulnerabilities of ECC

Although elliptic curve cryptography (ECC) is, as seen previously, very secure, several vulnerabilities can emerge. There exist attacks such as Baby Step Giant Step (BSGS) to solve the discrete logarithm problem. Their efficiency depends on the size of the group order, and for small curves, this attack can become feasible. The **Baby Step Giant Step** algorithm offers a time-memory tradeoff search method. The goal is to recover the secret integer k in the equation

$$Q = kG,$$

where G is a base point and Q is the result of the scalar multiplication of G by k.

Principle of the algorithm:

1. **Decomposition of k:** Let $m = \lceil \sqrt{n} \rceil$ where n is the group order. We then write $k = im + j$ with $0 \leq i, j < m$.

2. **Baby Steps:** Compute and store the values of jG for $j = 0, 1, 2, \ldots, m-1$.

3. **Giant Steps:** For $i = 0, 1, 2, \ldots, m-1$, compute

$$Q - i(mG)$$

and compare the result with the baby steps table. If for some i and j, we find

$$Q - i(mG) = jG,$$

then the secret exponent is obtained by $k = im + j$.

Although this approach remains efficient for small orders, it becomes impractical for large groups, thus guaranteeing the security of ECC systems in real-world applications. BSGS clearly demonstrates that recovering k from Q and G is made difficult by a quadratic growth in time and memory cost.

This also shows the importance of choosing the curve parameters. A subgroup of small size, a poorly chosen curve, or a bad choice of base point can compromise the security related to the discrete logarithm.

7 (Bonus) The Role of ECC in Bitcoin

Now that you know what elliptic curves over finite fields are and their interest in terms of cryptography, we will discuss their use in a real-world case: **Bitcoin**.

Bitcoin, put into circulation in 2009, is the first implementation of a 100% virtual and decentralized currency.

Bitcoin uses **ECC** for two main things: digital signatures and **Bitcoin** address generation.

7.1 Cryptographic Use of Bitcoin

7.1.1 Signature

Digital signatures ensure the authenticity and integrity of transactions in Bitcoin. This is done through the **ECDSA** algorithm on the special elliptic curve **secp256k1**.

When creating a new Bitcoin address, the user generates a private key on the **secp256k1** curve in the same way as we saw in the cryptography section. This private key will be used to sign Bitcoin transactions that the user makes with another person using **ECDSA**. From the signature we can ensure that the transaction is legitimate. The signature also allows anyone to verify the digital identity of the sending user without knowing their private key; this is done via their public key.

7.1.2 Address Generation

Bitcoin addresses are identifiers associated with public keys and **ECC** is involved in their generation.

From the private key we generate the user's public key as explained in the cryptography section; this 64-byte key is then passed to the **SHA-256** hash function which produces a 32-byte hash. The result is passed to a second hash function, **RIPEMD-160**, which results in a 20-byte hash. All of this is to create the most compact form possible of the public key. For the hashed address to be compatible with the Bitcoin network, the prefix **0x00** is added to the beginning of the public hash to signify that it is a standard **P2PKH** Bitcoin address.

Once we have obtained the hash with the prefix, there is still one step before the address is usable: we must compute the checksum and add it as a suffix to our address. It will be used to detect transmission errors and ensure that the address has not been modified or corrupted. To compute it, we hash our prefixed address with the **SHA-256** algorithm, then hash the result again with **SHA-256** and take the first 4 bytes of the final result to append as a suffix to our previously obtained prefixed address. After this entire complex process we finally have a valid Bitcoin address and it is encoded in **Base58Check** to make it readable to the user.

7.2 Choice of the secp256k1 Curve

The security of **Bitcoin** is due to the **ECC** it uses. The **secp256k1** curve, this finite field curve, was chosen for several reasons.

- **Security**: The complexity of the curve ensures that even with a 256-bit key, the security is extremely high.
- **Performance**: Good performance in signature computations. In particular, it allows faster computations for signing and verifying transactions, which is critical for the Bitcoin ecosystem to function.
- **Curve Properties**: Being a **Koblitz** curve (a = 0, b = 7), it has a particular mathematical structure that makes it better suited for the implementation of cryptographic algorithms used by **Bitcoin**.

7.3 Implementation

a, b, p, the generator point, and the curve's order are known for **secp256k1**, so we chose to implement a class that inherits from the Curve class but has hardcoded attributes.

```python
class SECP256k1(Curve):

    def __init__(self):


        super().__init__(a=0, b=7, p=0xfffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffefffffc2f)

        #Coordonnée du point générateur
        self.Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
        self.Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
        self.add_point((self.Gx,self.Gy))
        #Ordre de secp256k1
        self.ordre = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
        #Ordre du sous groupe cyclique généré (h = 1 donc il est le même que l'ordre de la courbe)
        self.n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

Figure 6: Implementation of the secp256k1 class in Python

A particularity of **secp256k1** is that the cofactor h equals 1, meaning that the cyclic subgroup generated by point G contains all the points needed for **secp256k1** cryptography.
The order of the cyclic subgroup is therefore the same as the curve's order since it equals $h * n$.

From this curve we proceeded to implement the functions necessary for creating a **Bitcoin** address as explained previously, using the **HashLib** library for hashing and checksum computation as well as the **base58** library for encoding. We obtain a Bitcoin address encoded in **base58** of the type

$$14v797gaVmnhUv8WYmc5efz2sdfqT8j8jV \tag{18}$$

Then we can create a Bitcoin transaction; to do this we take several pieces of information necessary for the proper execution of the transaction: the amount in BTC, the recipient's address, the previous transaction ID... and hash them together using **SHA256**.

Then we use the **ECDSA** algorithm to sign the transaction. To make **ECDSA** work we use the sender's private key d and the curve's order n. To sign a message m:

1. Compute $z = \text{hash}(m)$.
2. Generate a random number k.
3. Compute $r = (kG)_x \mod n$ (x-coordinate of point kG).
4. Compute $s = k^{-1}(z + rd) \mod n$.

The signature is the pair (r, s).

Then the signature is transmitted with the hashed message. The main purpose of the signature is to ensure that the message has not been modified since it was sent and to verify the digital identity of the sender. The receiver then applies the signature verification algorithm using the signature (r, s), the message m, and the sender's public key Q, still with the point G of the **secp256k1** curve.

1. Compute $z = \text{hash}(m)$.
2. Compute $w = s^{-1} \mod n$.
3. Compute $u_1 = zw \mod n$ and $u_2 = rw \mod n$.
4. Compute $P = u_1G + u_2Q$ (where $Q = dG$ is the public key).
5. The signature is valid if and only if $r \equiv P_x \mod n$.

7.4 Security of Software Wallets

Software wallets, not to be confused with hardware wallets like ledgers, are applications that allow storing, sending, and receiving cryptocurrencies using a Bitcoin address and with a graphical interface that makes them much more pleasant and practical to use. They can be computer-based, mobile app-based, or even web-based.

Although very practical, the major security flaw is that they store the private key on a device connected to the internet. The major security challenge is therefore the protection of the private key of all wallets stored on a software wallet. The risk is that if a malicious individual gains possession of the private key, they will be able to sign any transactions they want and send all the wallet's funds to other addresses.
To protect against this type of attack, online wallets have found several solutions.

- **Private key encryption**: Private keys are encrypted with algorithms like **AES-256** so that even if they are stolen they cannot be used.
- **Authentication**: They often use two-factor authentication protocols so that stealing the key alone does not allow its use.
- **Multi-signature**: Some wallets require multi-signature; multiple private keys are needed to sign and send a transaction.

8 Conclusion

Elliptic curves over finite fields have many real-world uses in message encryption and decryption. Starting only from addition and multiplication operations, we can implement many algorithms to generate private and public keys, but also encrypt, decrypt, sign, and even verify signatures.

The **Bitcoin** ecosystem relies entirely on a particular elliptic curve, **secp256k1**, which ensures the security of hundreds of thousands of daily transactions, transactions that can sometimes contain millions of euros.

It is the discrete logarithm problem that makes all of this possible: the practical impossibility of recovering a scalar k from P and kP on an elliptic curve guarantees the security of private keys. This fundamental property not only secures cryptocurrency wallets but also other cryptographic applications such as digital signature systems and key exchange protocols.

However, although elliptic curves offer an excellent level of security with shorter keys than classical schemes like **RSA**, they are not invulnerable. Advances in cryptanalysis and the emergence of quantum computing pose future challenges to these systems. Thus, research continues to explore new approaches, such as post-quantum cryptography, to ensure the security of cryptographic systems in the face of technological developments.

Elliptic curves therefore play a central role in modern cryptography and will continue to be studied and optimized to meet the growing needs for security and efficiency in an increasingly digital world.

References

[1] Fossé, S. (2025). Introduction to ECC. SAE S4 About Elliptic Curves.

[2] Bitcoin Wiki. `https://en.bitcoin.it/wiki/Main_Page`

[3] Python documentation on AES. `https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html`

[4] Python documentation on AES GCM mode. `https://pycryptodome.readthedocs.io/en/latest/src/cipher/modern.html#gcm-mode`

[5] ECDSA explained article. https://medium.com/coinmonks/ecdsa-the-art-of-cryptographic-signatures-d0bb2

[6] Bitcoin developer website. https://developer.bitcoin.org/

[7] Bitcoin stack exchange website. https://bitcoin.stackexchange.com/

[8] Computerphile, Elliptic curves https://www.youtube.com/watch?v=NF1pwjL9-DE

[9] Curves which make Bitcoin possible https://www.youtube.com/watch?v=qCafMW4OG7s

[10] Elliptic curves, scalar multiplication https://www.youtube.com/watch?v=A1mu2uuvAOs